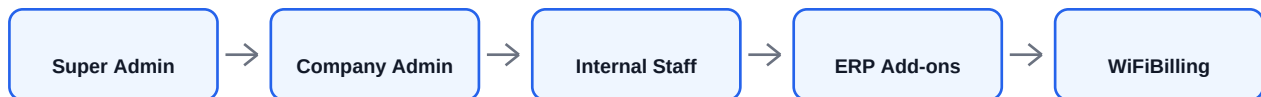


StudyRoom Connect ERP Roles, Permissions, Add-ons and Staff Assignment Guide

Implementation manual for role creation, add-on permission visibility, WiFiBilling permissions, and internal staff assignment.

Platform actor chain



Scope: Laravel ERP SaaS, Spatie Laravel Permission, Inertia React role pages, tenant-scoped staff users, and add-on-aware permission loading.

Generated for C:/xampp/htdocs/billing-system

Table of Contents

Page	Section
3	Executive Summary
4	Actor Model
5	Project Structure
6	MVC Request Flow
7	RoleController Permission Loading
8	Why WiFiBilling Permissions Were Hidden
9	Reusable Permission Loader
10	Role Create UI
11	Role Edit UI
12	Staff Assignment Model
13	User Creation Flow
14	User Edit Flow
15	Database and Model Relationships
16	Seeded Staff Profiles
17	Menu Visibility
18	Route Protection
19	Future Add-on Permission Steps
20	Troubleshooting
21	Cache Clearing
22	Verification Checklist
23	Affected Files
24	Implementation Sequence

Executive Summary



The roles and internal staff assignment system should work for the full ERP SaaS platform, not only for WiFi Billing. Super Admin owns the platform, Company/Admin owns a tenant, and Internal Staff receive tenant-scoped roles.

The root issue is usually not the React checkbox layout. The role pages render whatever grouped permissions the backend sends. If WiFiBilling permissions are inserted manually but do not appear, the visibility problem is normally in permission metadata, add-on activation, role ownership, or permission cache.

Key rule: a permission must be valid, assigned to the company/admin context, use the web guard, have a recognizable add_on and module, and pass the add-on activation filter before the role page can show it.

Actor Model

Access hierarchy

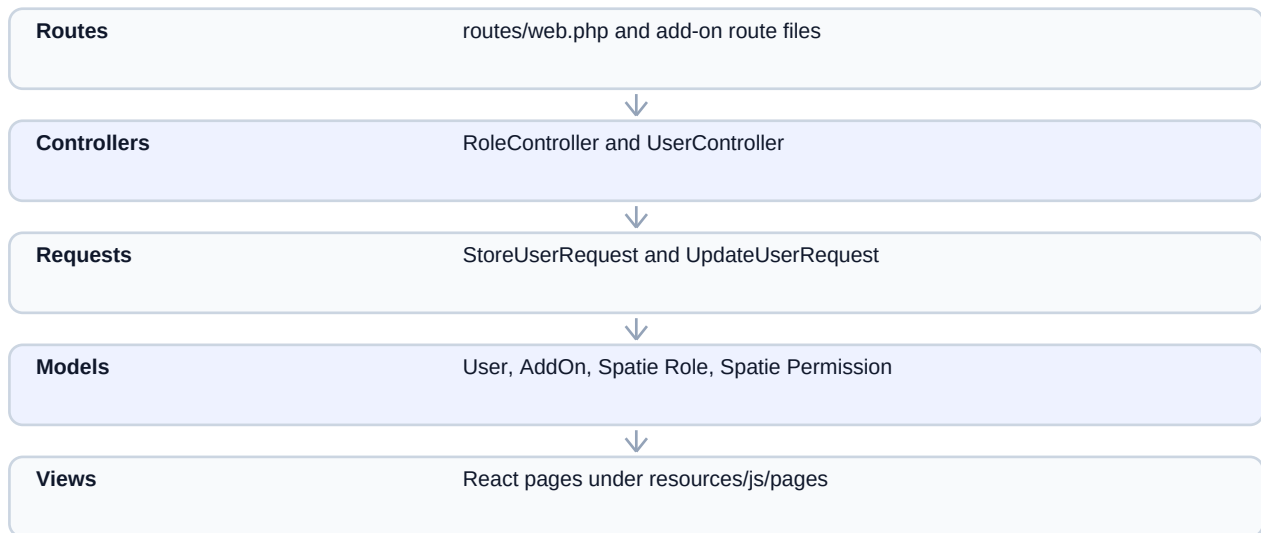


Super Admin is the platform owner and should not be mixed into tenant staff assignment. Company/Admin owns a tenant and creates roles for staff. Internal Staff users receive one or more tenant-scoped roles. Add-ons such as WiFiBilling contribute permissions to the tenant.

- Super Admin: controls plans, add-ons, company tenants, and platform settings.
- Company/Admin: manages business users, roles, modules, billing, and operations.
- Internal Staff: works inside a tenant with selected permissions.
- WiFiBilling: an ERP add-on that adds ISP billing, package, router, provisioning, and voucher permissions.

Project Structure

MVC and add-on structure

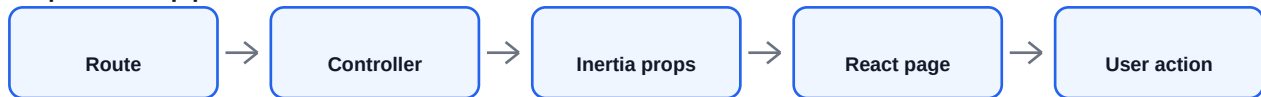


The files below are the main project areas involved in roles, permissions, add-ons, and staff assignment.

```
app/
  Http/Controllers/RoleController.php
  Http/Controllers/UserController.php
  Http/Requests/StoreUserRequest.php
  Http/Requests/UpdateUserRequest.php
  Models/User.php
  Models/AddOn.php
database/
  seeders/PermissionRoleSeeder.php
  seeders/IspRoleSeeder.php
resources/js/pages/
  roles/create.tsx
  roles/edit.tsx
  users/create.tsx
  users/edit.tsx
routes/
  web.php
packages/studyroomtechlab/WifiBilling/
  src/Routes/web.php
  src/Resources/js/menus/company-menu.ts
```

MVC Request Flow

Request to UI pipeline

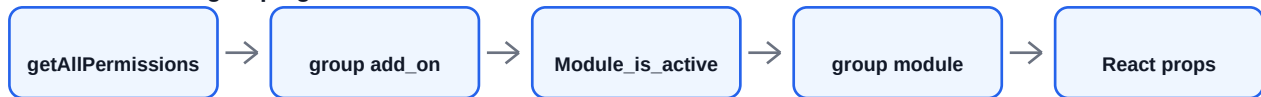


Laravel routes receive the browser request, controllers prepare tenant-scoped data, Inertia passes props to React, and React renders role or user forms. The role checkboxes are not hard-coded in the page; they are generated from the permission data sent by the controller.

- ``GET /roles/create`` maps to ``RoleController@create``.
- ``GET /roles/{role}/edit`` maps to ``RoleController@edit``.
- ``GET /users`` maps to ``UserController@index``, which passes assignable roles to user modals.
- ``POST /users`` maps to ``UserController@store``, which creates the user and assigns the selected role.

RoleController Permission Loading

Current backend grouping



The current controller calls `Auth::user()->getAllPermissions()` and then groups permissions by `'add_on'` and `'module'`. It also filters add-ons with `Module_is_active($addOn)`, except for the `'general'` group.

This design means a permission can exist in the database but still not render if the current company/admin does not have it, if the add-on is not active, or if the `'add_on'` value does not match the module registry.

```
$allPermissions = Auth::user()->getAllPermissions()  
    ->select('id', 'name', 'label', 'add_on', 'module');  
  
$permissions = $allPermissions->groupBy('add_on')  
    ->filter(fn ($items, $addOn) => $addOn === 'general' || Module_is_active($addOn))  
    ->map(fn ($items) => $items->groupBy('module'));
```

Why WiFiBilling Permissions Were Hidden

Visibility requirements



The likely failure is metadata and assignability, not the checkbox component. Manually inserted WiFiBilling rows need the same metadata and tenant visibility as seeded add-on permissions.

- ``guard_name`` must be ``web``.
- ``add_on`` must match the registered module key such as ``WiFiBilling``.
- ``module`` should be a readable module group such as ``mikrotik-routers`` or ``provisioning``.
- The company/admin role must have the permission or the loader must intentionally load assignable permissions beyond the current user's direct grants.
- ``Module_is_active('WiFiBilling')`` must return true for the tenant.
- Spatie permission cache may need reset after manual inserts.

Reusable Permission Loader

Create a reusable method or service for role permission loading. Both create and edit pages should use the same method. This avoids future add-ons disappearing because one controller action was updated and the other was not.

- Normalize empty `add\_on` to `Core`.
- Normalize empty `module` to `General`.
- Use `label` for display and `name` for submitted value.
- Filter out Super Admin only permissions when the current actor is Company/Admin.
- Preserve add-on grouping so the UI can render tabs.

```
private function assignablePermissions()
{
    return Permission::query()
        ->where('guard_name', 'web')
        ->get()
        ->filter(fn ($permission) => $this->permissionIsAssignable($permission))
        ->map(fn ($permission) => [
            'id' => $permission->id,
            'name' => $permission->name,
            'label' => $permission->label ?: Str::headline($permission->name),
            'add_on' => $permission->add_on ?: 'Core',
            'module' => $permission->module ?: 'General',
        ])
        ->groupBy('add_on')
        ->map(fn ($items) => $items->groupBy('module'));
}
```

Role Create UI

Create role UI structure

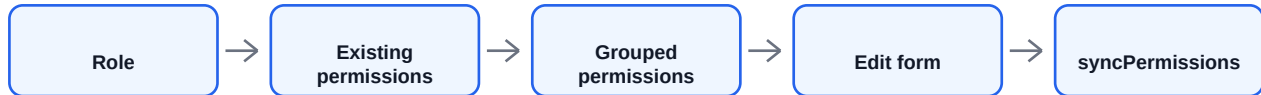


``resources/js/pages/roles/create.tsx`` renders add-on tabs and module cards from the ``permissions`` prop. The page already supports selecting a module through ``handleModuleChange``, but it should also search within module and permission labels and expose select-all actions for the whole add-on and all permissions.

- Add search across add-on alias, module name, permission label, and permission name.
- Add Select All for all visible permissions.
- Add Select Add-on for the current tab.
- Keep module checkbox indeterminate state.
- Keep submitted values as permission names.

Role Edit UI

Edit role lifecycle

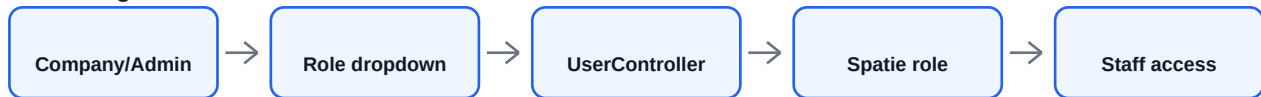


``resources/js/pages/roles/edit.tsx`` should mirror create behavior and preload ``rolePermissions``. If `WiFiBilling` appears in create but not edit, compare the props from ``RoleController@create`` and ``RoleController@edit``; both should call the same reusable loader.

- Use one shared frontend helper where practical.
- Preserve selected permissions from ``rolePermissions``.
- Support selecting and clearing a module after existing data loads.
- Respect non-editable role name rules while still allowing allowed permission edits where intended.

Staff Assignment Model

Staff assignment flow



Internal staff should be assigned a role/profile selected by the company/admin. The wording should be neutral ERP language: `Select a role/profile for this staff member. Permissions can be managed from Roles.`

Do not create a second role system. This project already uses Spatie Laravel Permission through `HasRoles`, `assignRole`, and `syncRoles`.

- Do not show Super Admin role in tenant staff forms.
- Load roles where `created\_by = creatorId()`.
- Validate that the selected role belongs to the same tenant.
- Use `assignRole` on create and `syncRoles` on edit.

User Creation Flow

Create internal staff sequence

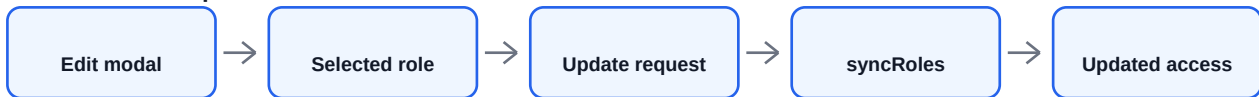


`UserController@store` currently reads a role id from the request field named `type`, creates the user, sets the `type` from the role name, and calls `\$user->assignRole(\$role)`. This is close to the desired behavior, but the field should be clearer as `role\_id` or `role\_ids` and should reject roles outside the tenant.

- Validate `role\_id` with an exists rule plus tenant ownership.
- Create the user with `created\_by = creatorId()` and `creator\_id = Auth::id()`.
- Set staff type from selected role or a stable internal type such as `staff`.
- Call `\$user->assignRole(\$role)` after save.

User Edit Flow

Edit staff role sequence



`resources/js/pages/users/edit.tsx` currently edits name, email, mobile number, and login status. For full staff role management, it should also load the user's current role and provide the same Role/Profile dropdown.

- Extend `EditUserData` with `role\_id` or `role\_ids`.
- Pass the current assigned role from `UserController@index` or a dedicated edit payload.
- Validate tenant ownership in `UpdateUserRequest`.
- Use `\$user->syncRoles([\$role])` for a single selected role.
- Keep Super Admin roles hidden and blocked on the backend.

Database and Model Relationships

Permission model stack



The project uses Spatie Laravel Permission. Users get roles through `model_has_roles`, and roles get permissions through `role_has_permissions`. The `User` model includes the `HasRoles` trait.

- `users.created_by` scopes a user to a tenant.
- `roles.created_by` scopes a role to a tenant.
- `permissions.add_on` groups permissions by add-on.
- `permissions.module` groups permissions inside each add-on.
- `user_active_modules` and plan modules affect add-on availability.

Seeded Staff Profiles

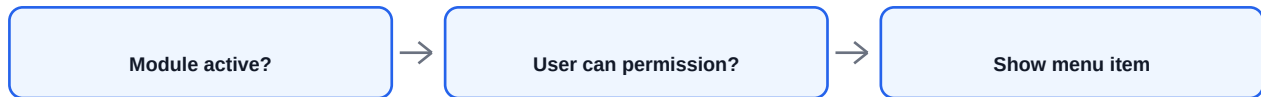
Seed common tenant roles safely. Use ``updateOrCreate`` or ``firstOrCreate`` with ``created_by`` so re-running the seeder does not duplicate roles.

- Admin
- Support Agent
- Billing Officer
- Sales / CRM
- Network Technician
- Installer
- Marketing
- Inventory Manager
- Viewer

```
Role::updateOrCreate(
    ['name' => 'network-technician', 'guard_name' => 'web', 'created_by' => $companyId],
    ['label' => 'Network Technician', 'editable' => true]
);
```

Menu Visibility

Menu decision



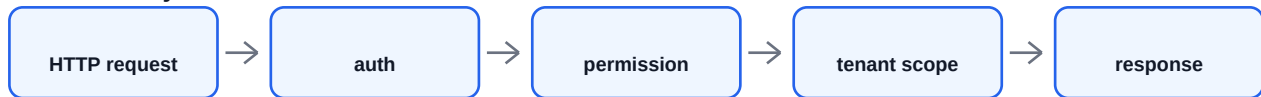
Menu visibility should be controlled by both add-on activation and permission checks. Hiding menu items improves UX, but route/controller checks are still required for security.

```
if (Module_is_active('WiFiBilling') && Auth::user()->can('view-mikrotik-routers')) {  
    // show MikroTik menu item  
}
```

- Company/Admin with full WiFiBilling permissions sees the module.
- Network Technician may see routers and provisioning only.
- Billing Officer may see packages, customers, invoices, and vouchers.
- Viewer may only see read-only dashboards.

Route Protection

Route security



Do not rely on React to protect data. Controllers or middleware must reject unauthorized users. Every route that exposes tenant data should check both permission and tenant ownership.

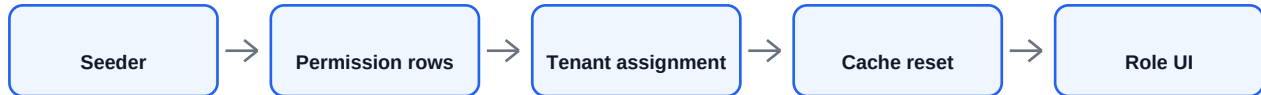
```
public function index()
{
  abort_unless(Auth::user()->can('view-mikrotik-routers'), 403);

  return MikrotikRouter::where('created_by', creatorId())->get();
}
```

Keep `/provision/{token}` public or token-protected according to its current design. Do not remove it while changing roles.

Future Add-on Permission Steps

Add-on lifecycle



Every future add-on should provide permissions through a repeatable installation/seeding path. The role pages should not need hard-coded edits for each add-on.

- Create add-on permission seeder.
- Set ``guard_name = web``.
- Set ``add_on`` to the exact module key.
- Set ``module`` to the feature group.
- Set a clear ``label``.
- Assign permissions to company/admin when the add-on is enabled.
- Clear permission cache.
- Confirm role create/edit render the new add-on tab.

Troubleshooting

Permission visibility decision tree



Use this checklist when permissions exist in the database but are not visible in the role page.

- Query the row and verify `name`, `guard\_name`, `label`, `add\_on`, and `module`.
- Check whether the current company/admin has the permission.
- Check `Module\_is\_active(add\_on)` for the current user.
- Check the add-on is installed and enabled.
- Check the active plan or user active modules include the add-on.
- Clear Laravel and Spatie permission caches.
- Inspect Inertia props for `permissions` on `/roles/create`.

Cache Clearing

Cache workflow



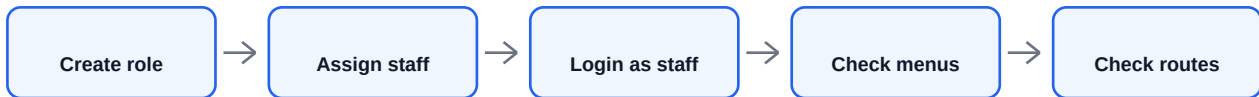
After changing permission seeders, manual permission rows, role assignments, or add-on activation, clear caches before testing.

```
C:\xampp\php\php.exe artisan optimize:clear  
C:\xampp\php\php.exe artisan config:clear  
C:\xampp\php\php.exe artisan permission:cache-reset
```

If `permission:cache-reset` is not available, continue after `optimize:clear` and `config:clear`. The command depends on the installed Spatie permission console commands.

Verification Checklist

End-to-end verification



Use this checklist after implementing the role and staff assignment fixes.

- ``/roles/create`` shows a WiFiBilling add-on group.
- ``/roles/edit`` shows the same WiFiBilling group.
- Network Technician role can be created.
- MikroTik, packages, provisioning, and voucher permissions can be selected.
- User creation shows Role/Profile, not fixed Support-only wording.
- Internal staff can be assigned Network Technician.
- Staff login only sees allowed menu items.
- Unauthorized routes return forbidden or redirect safely.
- Super Admin dashboard still works.
- ``/provision/{token}`` still works.
- PHP 8.2 syntax checks pass.

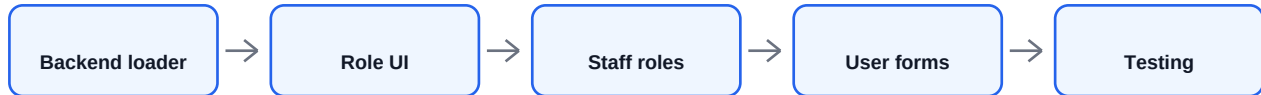
Affected Files

These files are the main places a developer should inspect or update for the roles, permissions, add-on visibility, and staff assignment work.

File path	Type	Why it matters	How to test
app/Http/Controllers/RoleController.php	Controller	Loads and groups permissions for create/edit role pages.	Open roles/create and inspect WiFiBilling groups.
resources/js/pages/roles/create.tsx	View	Renders add-on tabs, module boxes, and permission checkboxes.	Search and select permissions.
resources/js/pages/roles/edit.tsx	View	Edits existing role permissions.	Edit Network Technician.
app/Http/Controllers/UserController.php	Controller	Creates staff users and assigns selected Spatie roles.	Create staff and confirm model_has_roles.
resources/js/pages/users/create.tsx	View	Displays staff Role/Profile dropdown.	Create internal staff.
resources/js/pages/users/edit.tsx	View	Should allow changing staff role.	Edit staff and sync roles.
app/Http/Requests/StoreUserRequest.php	Request	Validates selected role on create.	Submit invalid role id.
app/Http/Requests/UpdateUserRequest.php	Request	Should validate selected role on edit.	Submit role from another tenant.
database/seeder/PermissionRoleSeeder.php	Seeder	Seeds core permissions and company roles.	Rerun seeder in test database.
packages/studyroomtechlab/WiFiBilling	Add-on	Contributes WiFi Billing routes, menus, and permissions.	Enable add-on and open role form.

Implementation Sequence

Recommended implementation order



A safe implementation order reduces regressions and makes testing easier.

- Step 1: Add or repair reusable permission loader.
- Step 2: Normalize permission metadata and add-on grouping.
- Step 3: Update role create/edit UI search and select-all behavior.
- Step 4: Seed staff profile roles for each company/admin safely.
- Step 5: Update user create/edit forms to use Role/Profile.
- Step 6: Validate role ownership on create and edit.
- Step 7: Assign or sync Spatie roles in UserController.
- Step 8: Clear caches and verify role pages.
- Step 9: Login as staff and confirm menus/routes.
- Step 10: Confirm Super Admin and provisioning route are untouched.